

OpenCTI Threat Intelligence Report

By: Utibe Anthony Akpan

Executive Summary

This project focused on extending an existing Security Operations Center (SOC) home lab by integrating **OpenCTI (Open Cyber Threat Intelligence Platform)** with **Wazuh** to introduce a structured threat intelligence layer into the existing security monitoring environment. The primary objective was to move beyond detecting isolated security events and enable the SOC environment to provide additional context by determining whether observed indicators were associated with known malicious activity, malware, threat actors, or MITRE ATT&CK techniques.

The implementation incorporated **OpenCTI**, **abuse.ch URLhaus**, **AlienVault Open Threat Exchange (OTX)**, and the **MITRE ATT&CK** knowledge base. URLhaus was used to provide malicious URL and IP indicators, while AlienVault OTX supplied community-driven threat intelligence and contextual relationships. MITRE ATT&CK provided standardized information about adversary behavior, techniques, malware, and intrusion sets. Due to the resource limitations of the laboratory environment, OpenCTI and Wazuh were configured with controlled resource usage rather than running all components simultaneously.

A custom Python integration, `custom-opencti.py`, was developed to bridge Wazuh alerts with the OpenCTI GraphQL API. When a monitored security event contained an IP address, the integration extracted the relevant indicator and queried OpenCTI for threat intelligence. Where a match was identified, the integration returned additional context such as the indicator's threat score, confidence level, labels, malware information, and MITRE ATT&CK context. Local Wazuh rules were then used to transform the enrichment response into a high-severity security alert that could be viewed within the Wazuh dashboard.

As part of the OpenCTI implementation, a **case study was applied to demonstrate the use of threat intelligence in an organizational context**. The research examined an organization and its relevant sector, identifying documented threats, attack activity, associated effects, threat actors, and other relevant threat information. The resulting information was represented within OpenCTI to demonstrate how organizational threat research can be structured and related to broader threat-intelligence data.

Several implementation challenges were encountered during development. These included memory contention caused by Elasticsearch and the OpenCTI Docker stack, Wazuh's restrictions on custom integration names, differences between Sysmon and Linux auditd event fields, non-persistent auditd rules, the absence of an appropriate Wazuh rule for auditd network events, integration-group configuration issues, and limited visibility into integration execution. Additional troubleshooting was required because successful delivery of enrichment data to the Wazuh socket did not automatically produce a dashboard alert; dedicated local Wazuh rules were required to process the returned JSON

The final implementation was validated using a genuine outbound connection to a known malicious URLhaus indicator. The connection triggered the custom integration, which successfully queried OpenCTI and identified the IP as a malicious indicator with a score of 80 and confidence of 100. The resulting Wazuh alert included additional context such as the **Mozi** malware label, ELF/MIPS characteristics, and MITRE ATT&CK information. The enrichment pipeline was subsequently applied to an existing SSH activity detection rule, demonstrating that threat intelligence could be incorporated into an existing SOC detection workflow without replacing the underlying detection logic.

Overall, the project demonstrated an end-to-end threat intelligence workflow in which external intelligence was ingested into OpenCTI, correlated with security events detected by Wazuh, enriched through the OpenCTI GraphQL API, and presented to an analyst through the SIEM. This provided a practical demonstration of how threat intelligence can transform a basic security alert into a more contextualized and actionable SOC event.

Scope & Objectives

Scope

The scope of this project covered the integration of **OpenCTI** into an existing Wazuh-based SOC home laboratory to provide structured threat intelligence enrichment for security events. The implementation focused on the collection, storage, querying, correlation, and presentation of threat intelligence within the existing security monitoring environment.

The project included the deployment and configuration of the OpenCTI platform and its supporting services using Docker Compose, integration of external threat intelligence sources, and connection of OpenCTI with the existing Wazuh environment. The selected intelligence sources were **abuse.ch URLhaus**, **AlienVault Open Threat Exchange (OTX)**, and **MITRE ATT&CK**.

The project also covered the development of a custom Python-based integration between Wazuh and the OpenCTI GraphQL API. The integration was designed to extract indicators from selected Wazuh security events, query OpenCTI for corresponding threat intelligence, and return relevant enrichment to Wazuh. Local Wazuh rules were subsequently configured to generate alerts containing the enrichment results.

Testing and validation were performed using Linux auditd network connection events and existing SSH activity detection within the laboratory environment. A known malicious URLhaus indicator was also used to validate the end-to-end enrichment workflow. The project further demonstrated the application of threat intelligence to an existing Wazuh detection rule.

An organizational threat-intelligence case study was also included to demonstrate how information about an organization's sector, threats, attack activity, threat actors, and associated effects can be represented and related within OpenCTI.

The project did not include penetration testing, exploitation of external systems, unauthorized scanning, or security assessment of third-party organizations. All technical validation was performed within the controlled laboratory environment or against known threat-intelligence indicators for the purpose of testing the enrichment workflow.

Objectives

The main objective of the project was to integrate OpenCTI with Wazuh to enhance security-event detection with structured and contextual threat intelligence.

The specific objectives were to:

1. **Deploy OpenCTI** within the existing SOC laboratory environment using Docker-based services.
2. **Integrate external threat intelligence sources** by connecting OpenCTI to URLhaus, AlienVault OTX, and MITRE ATT&CK.
3. **Develop a custom Wazuh-to-OpenCTI integration** capable of querying the OpenCTI GraphQL API when relevant security events are generated.
4. **Extract and correlate indicators** such as IP addresses from monitored security events with indicators stored in OpenCTI.
5. **Enrich Wazuh alerts** with threat scores, confidence levels, labels, malware context, and MITRE ATT&CK information when an indicator match is identified.
6. **Create dedicated Wazuh rules** capable of processing the enrichment response and generating context-rich security alerts.
7. **Apply threat intelligence enrichment to an existing SOC detection**, demonstrating how CTI can improve the context available to an analyst without replacing the underlying detection mechanism.
8. **Validate the complete enrichment workflow** from security-event generation through OpenCTI lookup to the final Wazuh alert.
9. **Document implementation challenges and their resolutions**, particularly those involving system resources, Wazuh integration behavior, auditd event structure, and alert processing.
10. **Demonstrate the practical use of OpenCTI for organizational threat intelligence** through a case study based on researched threat and sector information.

Methodology

Project Overview and Implementation Approach

The methodology for this project involved extending an existing Security Operations Center (SOC) home laboratory by introducing OpenCTI as a structured cyber threat intelligence platform and connecting it to the existing Wazuh SIEM environment. The implementation was designed to move beyond conventional event detection by allowing security alerts to be enriched with contextual threat intelligence, including malicious indicators, confidence scores, threat labels, and MITRE ATT&CK information.

The implementation was carried out across four major stages: deployment and configuration of OpenCTI, integration of external threat intelligence feeds, development of the Wazuh-to-OpenCTI enrichment pipeline, and application of the resulting intelligence to an existing SSH detection use case. Because the laboratory operated under limited hardware resources, the implementation was adapted to maintain a functional and repeatable environment while preserving the intended threat-intelligence workflow.

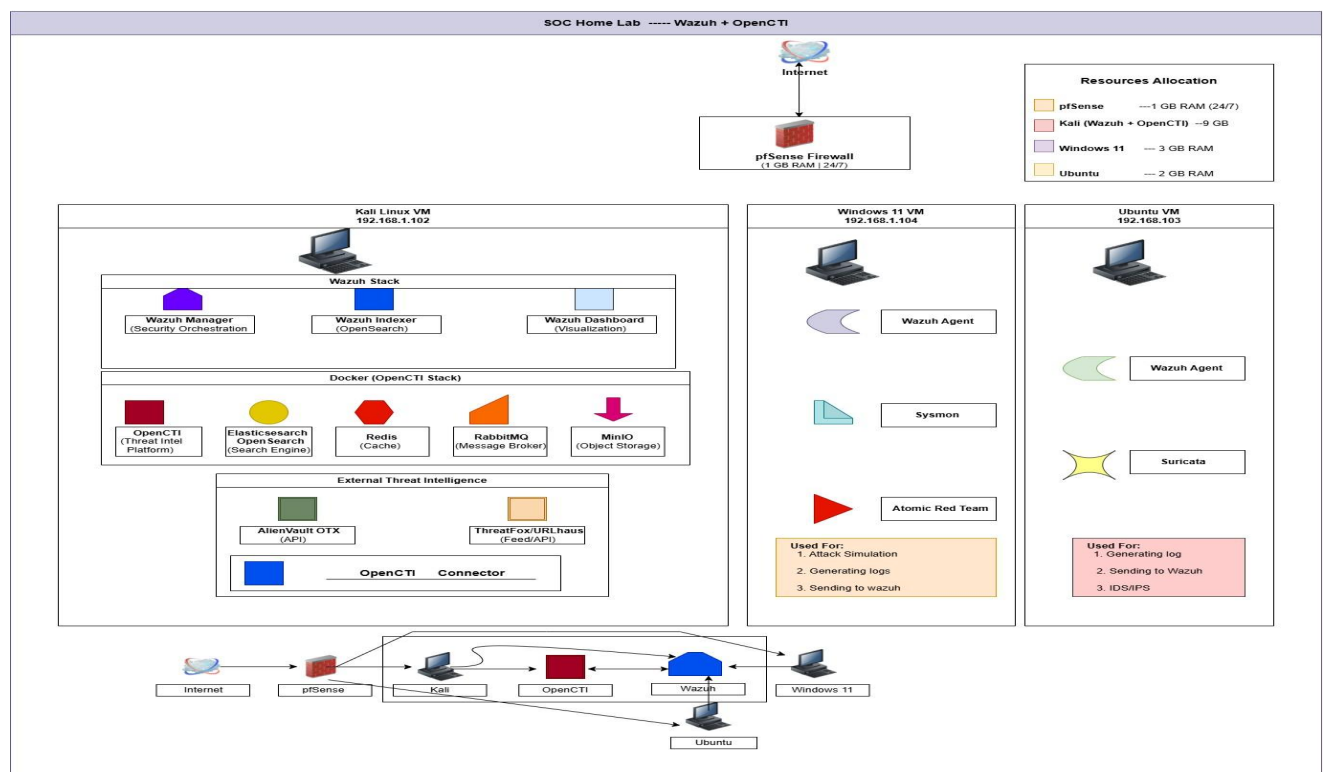
Laboratory Architecture and Environment

The project was implemented within the existing SOC laboratory environment. The primary system was a Kali Linux virtual machine allocated approximately 9 GB of RAM. Wazuh Manager was already deployed on the Kali system and was used as the SIEM component responsible for collecting, decoding, correlating, and generating security alerts. The Kali host was also registered locally as Wazuh agent 000, allowing its own security events to be monitored.

A separate pfSense virtual machine provided the virtual firewall and network-segmentation component of the laboratory. A Windows 11 environment with Sysmon was also available for endpoint telemetry; however, running the Windows VM together with Wazuh and the OpenCTI Docker environment placed significant memory pressure on the host. Consequently, Linux auditd was used as the primary network-event source during the final enrichment testing, while the Sysmon processing branch was retained within the integration logic.

OpenCTI was deployed as a Docker Compose stack on the Kali system. The deployment included the OpenCTI platform together with its supporting services: Elasticsearch for data storage and search, Redis for caching, RabbitMQ for message brokering, and MinIO for object storage. These components operated together to provide the underlying services required by the OpenCTI platform.

The resulting architecture connected external threat-intelligence sources to OpenCTI, where indicators and associated intelligence were stored and correlated. Wazuh-generated security alerts were then passed to a custom Python integration, which queried OpenCTI through its GraphQL API. When a matching indicator was identified, the integration returned the threat-intelligence context to Wazuh, where dedicated rules converted the enrichment data into a high-severity alert.



Tools and Technologies

The implementation used a combination of SIEM, threat-intelligence, endpoint-monitoring, containerization, and scripting technologies. Wazuh provided log collection, decoding, rule-based detection, and alert generation. OpenCTI provided the central threat-intelligence knowledge base and relationship-based representation of indicators, threat actors, malware, and MITRE ATT&CK information.

Docker Compose was used to deploy and manage the OpenCTI platform and its supporting services. Elasticsearch provided the OpenCTI search and storage backend, Redis provided caching, RabbitMQ handled message queuing, and MinIO provided object storage.

Threat intelligence was obtained from **abuse.ch URLhaus** and **AlienVault OTX**, while the built-in **MITRE ATT&CK** connector supplied structured adversary and technique information. Python was used to develop the `custom-opencti.py` integration, while the OpenCTI GraphQL API provided the interface through which Wazuh alerts were queried against the threat-intelligence database.

For endpoint telemetry, Sysmon was used for Windows event collection where the Windows environment was available, while Linux `auditd` was used to capture `network connect()` system calls during the final testing process.

Phase One — OpenCTI Deployment and Initial Configuration

The first implementation stage involved deploying OpenCTI within the existing Kali Linux environment. Rather than creating a new virtual machine, the existing SOC laboratory was extended by installing OpenCTI through Docker Compose. This approach allowed the threat-intelligence platform to operate alongside the existing Wazuh environment while avoiding the additional storage and infrastructure requirements associated with deploying another virtual machine.

The Docker environment was prepared and the OpenCTI Compose deployment was used to bring up the required platform and supporting services. The OpenCTI platform depended on Elasticsearch, Redis, RabbitMQ, and MinIO, which were deployed as part of the stack.

Because the available memory was limited, the OpenCTI services were not continuously operated alongside the full Wazuh environment. During periods when OpenCTI was required, the relevant services were started, while Wazuh could be stopped when necessary to reduce resource contention.

```
kali@kali: ~/Desktop
Session Actions Edit View Help

sudo docker run hello-world
Docker version 28.5.2+dfsg4, build 9cc6dea35e9a963f281434761c656fba4c43aed
Docker Compose version 2.40.3-3
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:5e23090353324d887c48ad5e5c56d294eab81588df9605b07d1afe895f9cc8f8
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

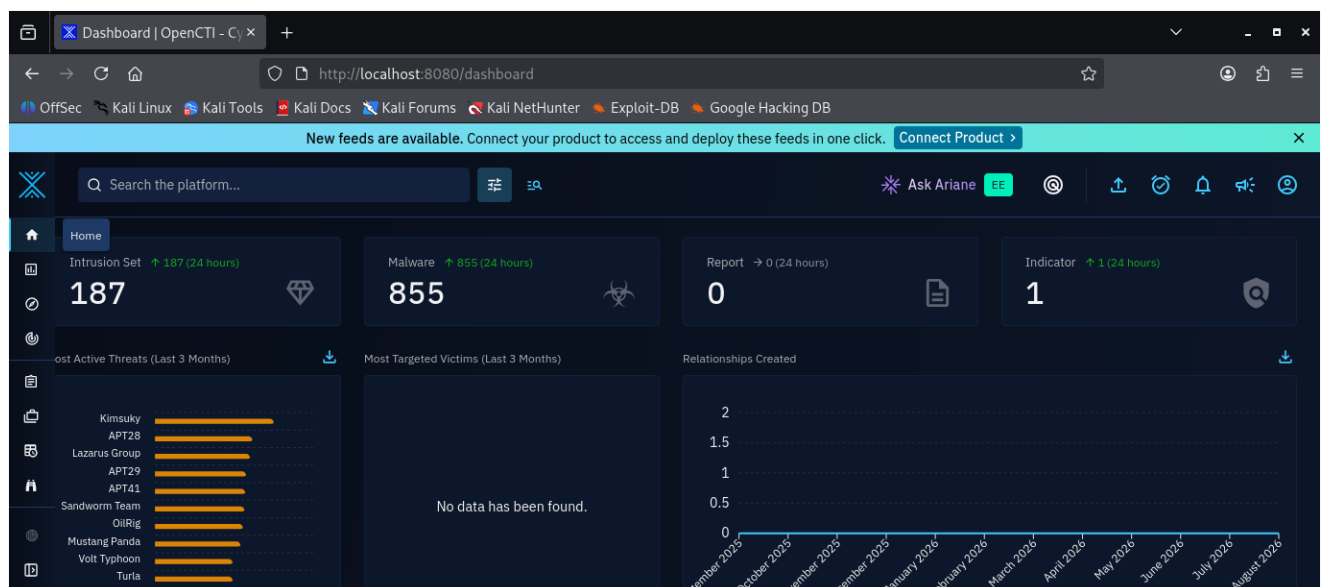
For more examples and ideas, visit:
https://docs.docker.com/get-started/

(kali@kali) - [~/Desktop]
$
```

The OpenCTI platform was then accessed through its web interface and used as the central location for managing the threat-intelligence data introduced during the project. The platform's knowledge graph provided the environment in which indicators, threat actors, malware, intrusion sets, and ATT&CK relationships could be represented and queried.

The built-in MITRE ATT&CK connector was subsequently enabled to populate the platform with ATT&CK knowledge. After ingestion, the OpenCTI environment contained ATT&CK-related intrusion sets and malware objects that could be associated with indicators and other intelligence objects.

The resulting ATT&CK dataset included **187 Intrusion Sets** and **855 Malware objects**, providing the threatintelligence foundation used later during enrichment and analysis.



Phase Two — Threat Intelligence Feed Integration

The second stage introduced external threat-intelligence sources into OpenCTI. Two feeds were integrated: **abuse.ch URLhaus** and **AlienVault OTX**. The feeds were selected to provide complementary intelligence, with URLhaus supplying malicious URL and associated indicator information and OTX providing broader community-contributed threat pulses and contextual relationships.

URLhaus Integration

The abuse.ch URLhaus connector was configured within OpenCTI to ingest malicious URL intelligence. The feed provided indicators associated with URLs used to distribute malware, together with related information such as IP addresses and hashes.

Following configuration, the connector was allowed to ingest intelligence into OpenCTI. The resulting data provided indicators that could subsequently be queried by the Wazuh enrichment integration.

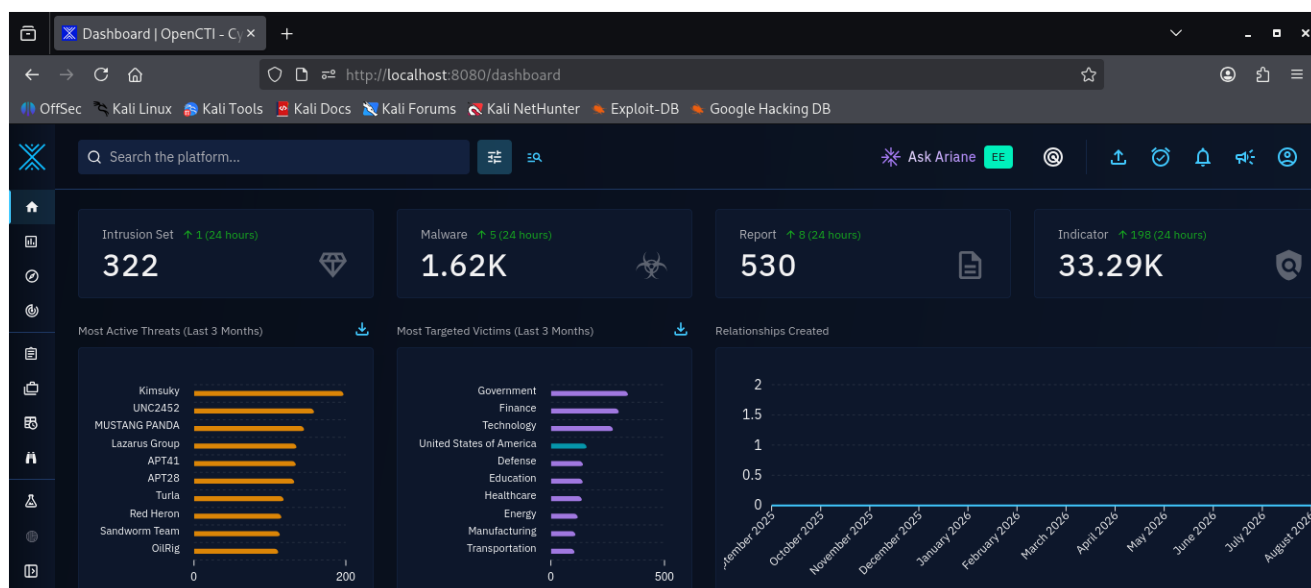
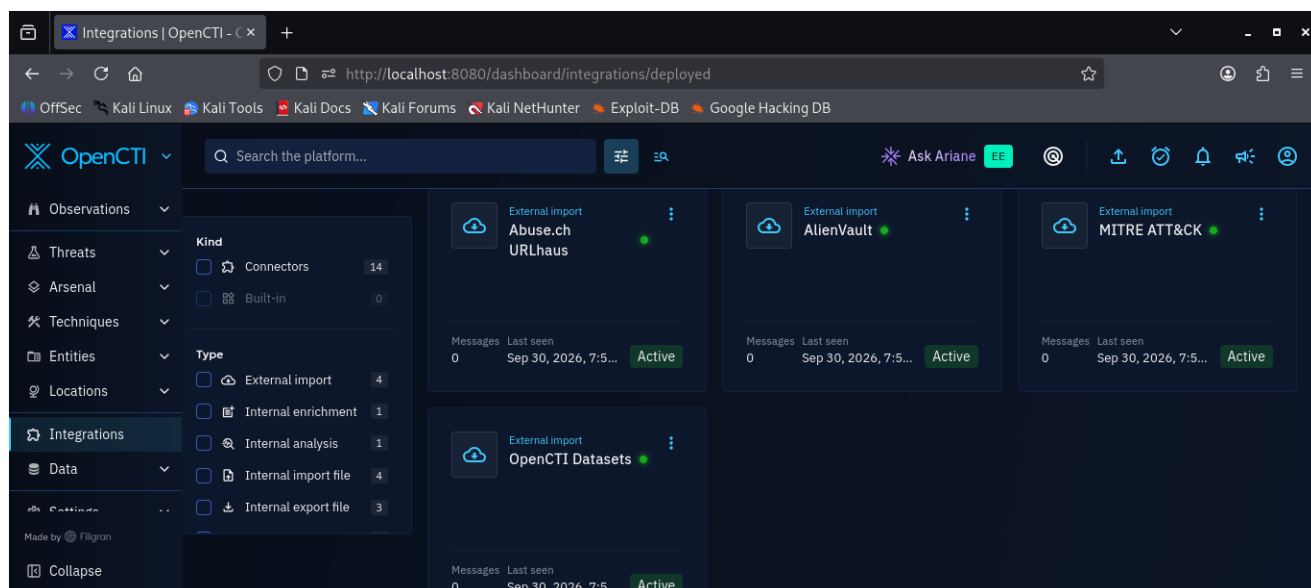
Approximately **900+ indicators** were ingested from URLhaus during the project.

AlienVault OTX Integration

AlienVault OTX was configured as the second external intelligence source. Unlike URLhaus, which primarily supplied individual malicious indicators, OTX provided community-created threat pulses containing groups of related indicators and contextual information.

The connector successfully imported OTX intelligence into OpenCTI. A single pull produced approximately **99 pulses and approximately 492,000 operations**, substantially increasing the volume of data processed by the platform.

The imported intelligence also provided relationships to MITRE ATT&CK techniques. During ingestion, relationships involving techniques such as **T1070.004 (File and Directory Discovery)** and **T1564.001 (Hidden Files and Directories)** were observed in the connector processing information.



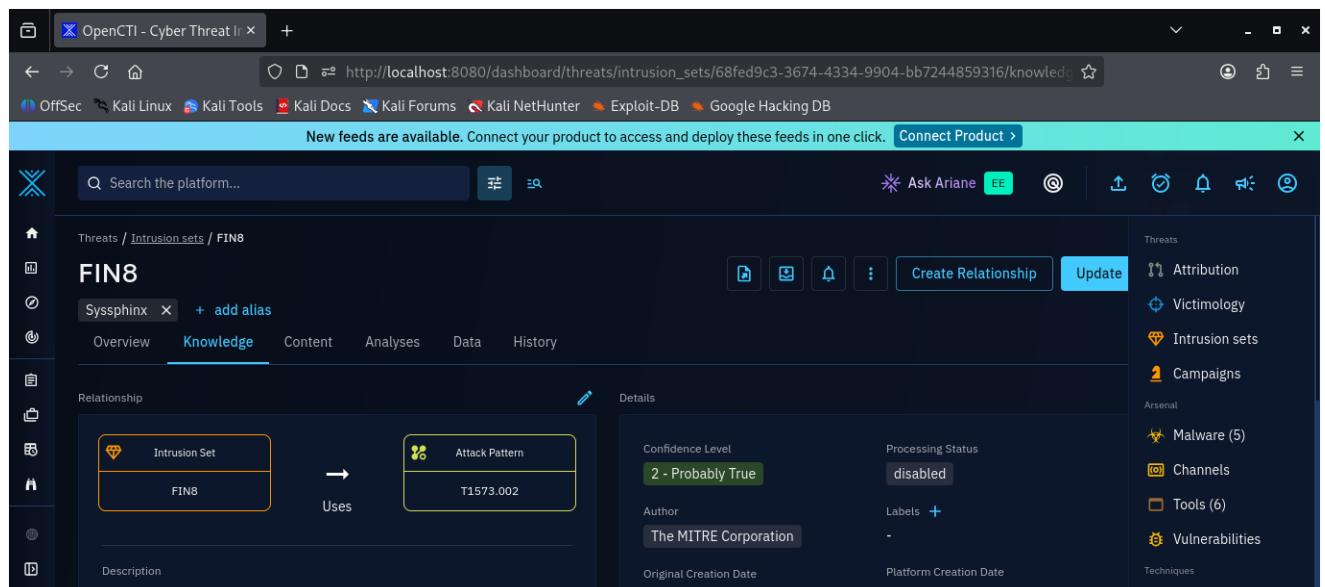
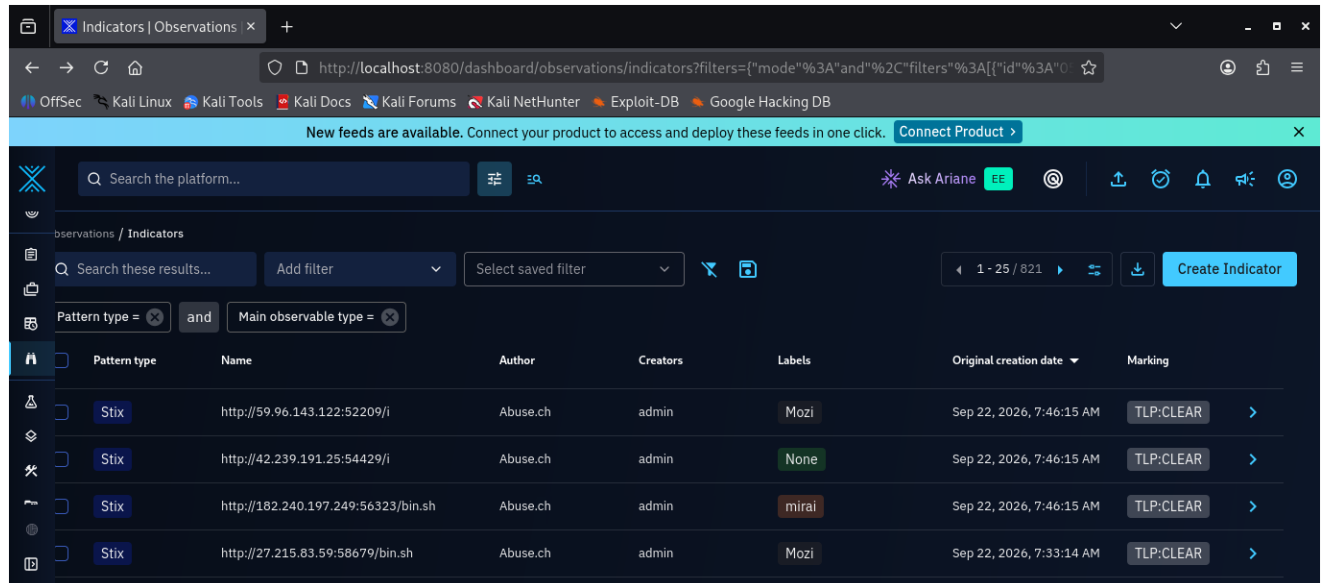
The volume of OTX data also contributed to the resource pressure experienced by the laboratory. The project therefore continued with the two active feeds rather than introducing an additional feed such as ThreatFox, allowing the available system resources to remain manageable.

Phase Three — Wazuh and OpenCTI Enrichment Pipeline

The third stage formed the technical core of the implementation. The objective was to establish a mechanism through which Wazuh alerts could automatically query OpenCTI and receive threat-intelligence context when a relevant indicator was detected.

The integration was implemented using a custom Python script named `custom-opencti.py`. Rather than relying on a manual lookup process, the script provided an automated connection between Wazuh and OpenCTI through the OpenCTI GraphQL API.

The workflow was structured so that Wazuh generated an alert from monitored activity, the integration extracted the relevant IP address, and the extracted value was submitted to OpenCTI for matching. When OpenCTI returned a matching indicator, the script extracted the available intelligence, including confidence, score, and labels, and returned an enrichment message into Wazuh.



Initial Sysmon-Based Approach

The original implementation was designed around Windows Sysmon Event ID 3, which provides networkconnection telemetry. The integration therefore initially expected the destination IP to be available through Wazuh's structured Sysmon event fields.

This approach was subsequently adapted because simultaneously operating Kali, Wazuh, OpenCTI's Docker environment, pfSense, and the Windows VM placed excessive memory pressure on the laboratory host. Rather than discontinue live testing, the network-event source was changed to Linux auditd on the Kali host itself.

The Sysmon processing branch remained within the Python integration, allowing the same enrichment logic to support the Windows telemetry path when the Windows environment is available.

Auditd Network Monitoring

Linux auditd was configured to monitor network `connect()` system calls using the `network_connect` audit key. This provided a local source of network connection events without requiring the Windows VM to remain active during OpenCTI testing.

The audit events were collected by Wazuh and passed through the auditd decoder. Because the auditd network information was contained within the raw event rather than exposed through the same structured fields available from Sysmon, the integration was extended with a separate extraction method.

The `custom-opencti.py` script therefore used a regular expression against the auditd `full_log` content to extract the destination IP address and port from the `saddr_fam`, `laddr`, and `lport` values.



```
kali@kali: ~/Desktop
Session Actions Edit View Help

(kali@kali)~$ sudo apt install -y auditd audispd-plugins
The following packages were automatically installed and are no longer required:
libicu76 libipc-run-perl libsvtavlc2 libzxing3 openjdk-21-jre-headless python3-pysnmp4
libio-pty-perl libsimdtf31 libteamctl0 openjdk-21-jre postgresql-common-dev
Use 'sudo apt autoremove' to remove them.

Upgrading:
libaudit1
Installing:
audispd-plugins auditd
Installing dependencies:
libauparse0t64 libauplugin1
Summary:
Upgrading: 1, Installing: 4, Removing: 0, Not Upgrading: 1875
Download size: 445 kB
Space needed: 1,229 kB / 30.6 GB available

Get:1 http://http.kali.org/kali kali-rolling/main amd64 libaudit1 amd64 1:4.1.2-1+b2 [59.1 kB]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 libauparse0t64 amd64 1:4.1.2-1+b2 [74.8 kB]
Get:3 http://http.kali.org/kali kali-rolling/main amd64 libauplugin1 amd64 1:4.1.2-1+b2 [21.4 kB]
Get:4 http://http.kali.org/kali kali-rolling/main amd64 auditd amd64 1:4.1.2-1+b2 [239 kB]
Get:5 http://http.kali.org/kali kali-rolling/main amd64 audispd-plugins amd64 1:4.1.2-1+b2 [51.0 kB]
Fetched 445 kB in 4s (118 kB/s)
(Reading database... 549144 files and directories currently installed.)
Preparing to unpack ./libaudit1_1%3a4.1.2-1+b2_amd64.deb...
Unpacking libaudit1:amd64 (1:4.1.2-1+b2) over (1:4.1.2-1+b1)...
Setting up libaudit1:amd64 (1:4.1.2-1+b2)...
Selecting previously unselected package libauparse0t64:amd64.
```

```
kali@kali: ~/Desktop
Session Actions Edit View Help
exe=/opt/erlang/lib/erlang/erts-15.2.7.13/bin/inet_ghost subj=docker-default key=network_connect
type=SOCKADDR msg=audit(09/25/2026 12:13:04.789:1103) : saddr={ saddr_fam=inet laddr=127.0.0.1 lport=4369 }
type=PROCTITLE msg=audit(09/25/2026 12:13:04.805:1104) : proctitle=/opt/erlang/lib/erlang/erts-15.2.7.13/bin/inet_ghost 4
type=SYSCALL msg=audit(09/25/2026 12:13:04.805:1104) : arch=x86_64 syscall=connect success=no exit=ENOENT(No such file or directory) a0=0x3 a1=0x7fffd876f480 a2=0x6e
a3=0x1 items=1 ppid=78963 pid=78964 auid=unset uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=unset comm=inet_ghost
exe=/opt/erlang/lib/erlang/erts-15.2.7.13/bin/inet_ghost subj=docker-default key=network_connect
type=PROCTITLE msg=audit(09/25/2026 12:13:04.805:1105) : proctitle=/opt/erlang/lib/erlang/erts-15.2.7.13/bin/inet_ghost 4
type=SYSCALL msg=audit(09/25/2026 12:13:04.805:1105) : arch=x86_64 syscall=connect success=no exit=ENOENT(No such file or directory) a0=0x3 a1=0x7fffd876f670 a2=0x6e
a3=0x1 items=1 ppid=78963 pid=78964 auid=unset uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=unset comm=inet_ghost
exe=/opt/erlang/lib/erlang/erts-15.2.7.13/bin/inet_ghost subj=docker-default key=network_connect
type=SOCKADDR msg=audit(09/25/2026 12:13:04.809:1106) : saddr={ saddr_fam=inet laddr=127.0.0.1 lport=4369 }
type=SOCKADDR msg=audit(09/25/2026 12:13:04.809:1107) : saddr={ saddr_fam=inet laddr=127.0.0.1 lport=25672 }
type=SOCKADDR msg=audit(09/25/2026 12:13:05.389:1108) : saddr={ saddr_fam=inet laddr=127.0.0.1 lport=6379 }
type=SOCKADDR msg=audit(09/25/2026 12:13:05.389:1109) : saddr={ saddr_fam=inet laddr=127.0.0.1 lport=6379 }

(kali@kali) ~/Desktop
$ sudo grep -a "123.11.125.96" /var/ossec/logs/archives/archives.json | tail -5

(kali@kali) ~/Desktop
$ sudo grep -a "123.11.125.96" /var/ossec/logs/archives/archives.json | tail -5
{"timestamp": "2026-09-25T07:16:32.260-0400", "rule": {"level": 3, "description": "Successful sudo to ROOT executed.", "id": "5402", "mitre": {"id": "T1548.003", "tactic": ["Privilege Escalation", "Defense Evasion"], "technique": ["Sudo and Sudo Caching"]}, "firedtimes": 4, "mail": false, "groups": ["syslog", "sudo"], "pci_dss": ["10.2.5", "10.2.2"], "gpg13": ["7.6", "7.8", "7.13"], "gdpr": ["IV.32.2"], "hipaa": ["164.312.b"], "nist_800_53": ["AU.14", "AC.7", "AC.6"], "tsc": ["CC6.8", "CC7.2", "CC7.3"]}, "agent": {"id": "000", "name": "kali"}, "manager": {"name": "kali"}, "id": "1790334992.118102", "full_log": "Sep 25 11:16:30 kali sudo[83105]: kali : TTY=pts/0 ; PWD=/home/kali/Desktop ; USER=root ; COMMAND=/usr/bin/grep -a 123.11.125.96 /var/ossec/logs/archives/archives.json", "predecoder": {"program_name": "sudo", "timestamp": "Sep 25 11:16:30", "hostname": "kali", "decoder": {"parent": "sudo", "name": "sudo", "ftscomment": "First time user executed the sudo command", "data": {"srcuser": "kali", "dstuser": "root", "tty": "pts/0", "pwd": "/home/kali/Desktop", "command": "/usr/bin/grep -a 123.11.125.96 /var/ossec/logs/archives/archives.json"}, "location": "journal"}}
```

Creating the Wazuh Alert for Network Events

Although auditd was successfully generating and Wazuh was successfully decoding the network events, the events initially did not appear as actionable alerts because they did not have a rule assigning them an alert level.

The custom rule was subsequently created as rule **100100**. The rule was structured as a child of Wazuh's existing audit rule **80700** and used the `network_connect` audit key to identify the relevant events. This allowed the network-connection activity to receive a level-5 alert and become available to the integration process.

```
kali@kali: ~/opencti-docker
Session Actions Edit View Help

(kali@kali) ~/opencti-docker
$ sudo chown root:wazuh /var/ossec/integrations/custom-opencti.py

(kali@kali) ~/opencti-docker
$ sudo /var/ossec/framework/python/bin/python2 /var/ossec/integrations/custom-opencti.py /tmp/test_alert.json "b7376aa2-5f80-4536-b699-c87a30ab8417" "" debug

(kali@kali) ~/opencti-docker
$ sudo cat /var/ossec/logs/integrations.log | tail -5
# No usable destination IP found
# Processing alert: 1790463664.39576552
# 172.18.0.10 is not a public IP, skipping
# Processing alert: 1790463666.39577963
# No usable destination IP found

(kali@kali) ~/opencti-docker
$ sudo grep -a -A2 "123.11.125.96" /var/ossec/logs/integrations.log | tail -20
# Querying OpenCTI for IP: 123.11.125.96
# Processing alert: 1790432507.9427465
# Querying OpenCTI for IP: 123.11.125.96
# Processing alert: 1790463495.35984395
# No usable destination IP found
--
# Querying OpenCTI for IP: 123.11.125.96
# MATCH FOUND, sending: {'integration': 'opencti', 'opencti': {'found': 1, 'destination_ip': '123.11.125.96', 'score': 80, 'confidence': 100, 'pattern': "[url:value = 'http://123.11.125.96:40919/bin.sh']", 'labels': ['32-bit', 'Mozi', 'elf', 'financial-sector-relevant', 'mips']}}
# send_msg called with: {'integration': 'opencti', 'opencti': {'found': 1, 'destination_ip': '123.11.125.96', 'score': 80, 'confidence': 100, 'pattern': "[url:value = 'http://123.11.125.96:40919/bin.sh']", 'labels': ['32-bit', 'Mozi', 'elf', 'financial-sector-relevant', 'mips']}}
# Processing alert: 1790463664.39571957
# 127.0.0.11 is not a public IP, skipping

(kali@kali) ~/opencti-docker
$
```


OpenCTI GraphQL Query and Enrichment

When invoked, `custom-opencti.py` received the Wazuh alert and extracted the relevant IP address. The IP address was then submitted to OpenCTI through its GraphQL API.

When a matching indicator was found, the script retrieved the associated threat-intelligence information. The returned information included the OpenCTI score, confidence value, and labels associated with the indicator.

The enrichment information was then formatted into a JSON message and sent back to Wazuh through its Unix integration socket.

Converting the Enrichment Message into a Wazuh Alert

Sending the enrichment message to Wazuh's Unix socket did not automatically create a visible security alert. Dedicated local Wazuh rules were therefore created to interpret the JSON enrichment message.

Rules **100150–100152** were introduced to process the OpenCTI integration output. The rules included a base integration rule, a no-match condition, and a high-severity rule for a successful malicious-indicator match.

The successful match rule generated a **level-12** alert and incorporated the destination IP and OpenCTI score into the alert description.

Integration Debugging and Verification

During development, additional visibility into the integration process was enabled by increasing Wazuh's `integrator.debug` value to **2** in `internal_options.conf`. This allowed the integration process to record its execution steps, including alert receipt, IP extraction, OpenCTI queries, match results, and socket delivery.

The resulting integration logs provided confirmation that the script was being invoked correctly and that OpenCTI queries were returning the expected information.

```
kali@kali: ~/Desktop
Session Actions Edit View Help
exe=/opt/erlang/lib/erlang/erts-15.2.7.13/bin/inet_gethost subj=docker-default key=network_connect
type=SOCKADDR msg=audit(09/25/2026 12:13:04.789:1103) : saddr={ saddr_fam=inet laddr=127.0.0.1 lport=4369 }
type=PROCTITLE msg=audit(09/25/2026 12:13:04.805:1104) : proctitle=/opt/erlang/lib/erlang/erts-15.2.7.13/bin/inet_gethost 4
type=SYSCALL msg=audit(09/25/2026 12:13:04.805:1104) : arch=x86_64 syscall=connect success=no exit=ENOENT(No such file or directory) a0=0x3 a1=0x7fffd876f480 a2=0x6e
a3=0x1 items=1 ppid=78963 pid=78964 auid=unset uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=unset comm=inet_gethost
exe=/opt/erlang/lib/erlang/erts-15.2.7.13/bin/inet_gethost subj=docker-default key=network_connect
type=PROCTITLE msg=audit(09/25/2026 12:13:04.805:1105) : proctitle=/opt/erlang/lib/erlang/erts-15.2.7.13/bin/inet_gethost 4
type=SYSCALL msg=audit(09/25/2026 12:13:04.805:1105) : arch=x86_64 syscall=connect success=no exit=ENOENT(No such file or directory) a0=0x3 a1=0x7fffd876f670 a2=0x6e
a3=0x1 items=1 ppid=78963 pid=78964 auid=unset uid=root gid=root euid=root suid=root fsuid=root egid=root sgid=root fsgid=root tty=(none) ses=unset comm=inet_gethost
exe=/opt/erlang/lib/erlang/erts-15.2.7.13/bin/inet_gethost subj=docker-default key=network_connect
type=SOCKADDR msg=audit(09/25/2026 12:13:04.809:1106) : saddr={ saddr_fam=inet laddr=127.0.0.1 lport=4369 }
type=SOCKADDR msg=audit(09/25/2026 12:13:04.809:1107) : saddr={ saddr_fam=inet laddr=127.0.0.1 lport=25672 }
type=SOCKADDR msg=audit(09/25/2026 12:13:05.389:1108) : saddr={ saddr_fam=inet laddr=127.0.0.1 lport=6379 }
type=SOCKADDR msg=audit(09/25/2026 12:13:05.389:1109) : saddr={ saddr_fam=inet laddr=127.0.0.1 lport=6379 }

(kali@kali) ~/Desktop
$ sudo grep -a "123.11.125.96" /var/ossec/logs/archives/archives.json | tail -5
(kali@kali) ~/Desktop
$ sudo grep -a "123.11.125.96" /var/ossec/logs/archives/archives.json | tail -5
{"timestamp": "2026-09-25T07:16:32.260-0400", "rule": {"level": 3, "description": "Successful sudo to ROOT executed.", "id": "5402", "mitre": [{"id": "T1548.003"], "tactic": ["Privilege Escalation", "Defense Evasion"]}, "technique": ["Sudo and Sudo Caching"]}, "firedtimes": 4, "mail": false, "groups": ["syslog", "sudo"], "pci_dss": ["10.2.5", "10.2.2"], "gpg13": [{"7.6", "7.8", "7.13"}], "gdpr": ["IV.32.2"]}, "hipaa": ["164.312.b"], "nist_800_53": ["AU.14", "AC.7", "AC.6"]}, "tsc": [{"CC6.8", "CC7.2", "CC7.3"}], "agent": {"id": "000", "name": "kali"}, "manager": {"name": "kali"}, "id": "1790334992.118102", "full_log": "Sep 25 11:16:30 kali sudo[83105]: kali : TTY=pts/0 ; PWD=/home/kali/Desktop ; USER=root ; COMMAND=/usr/bin/grep -a 123.11.125.96 /var/ossec/logs/archives/archives.json", "predecoder": {"program_name": "sudo", "timestamp": "Sep 25 11:16:30", "hostname": "kali"}, "decoder": {"parent": "sudo", "name": "sudo", "ftsccomment": "First time user executed the sudo command", "data": {"srcuser": "kali", "dstuser": "root", "tty": "pts/0", "pwd": "/home/kali/Desktop", "command": "/usr/bin/grep -a 123.11.125.96 /var/ossec/logs/archives/archives.json"}, "location": "journal"}}
```

Phase Four — Applied Detection Use Case

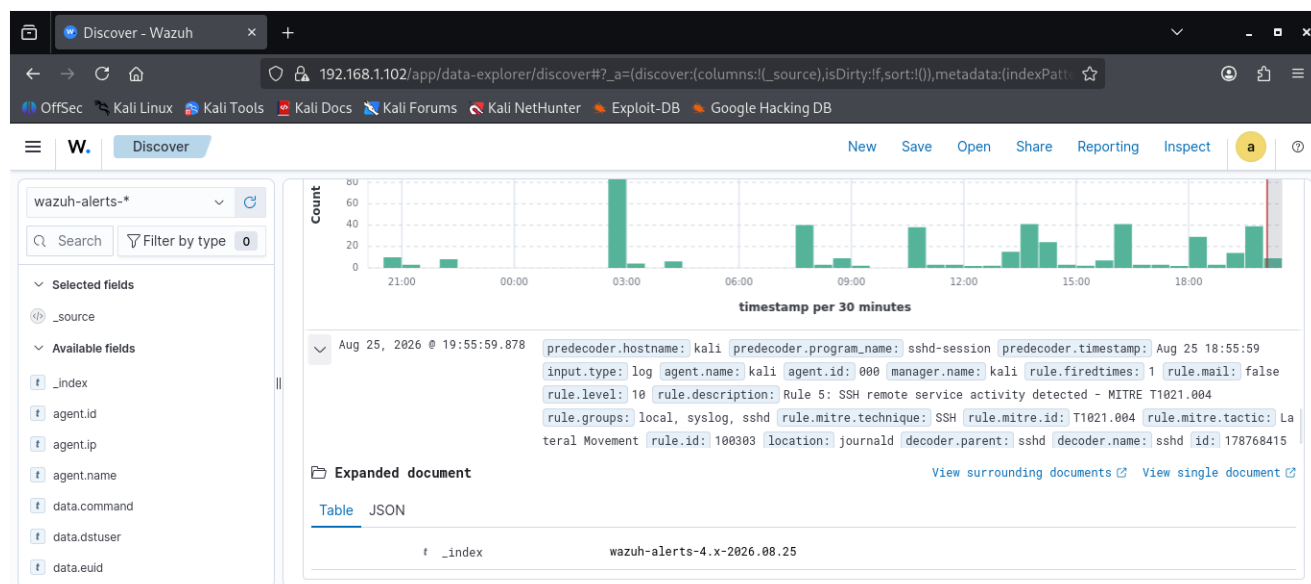
After the enrichment pipeline was operational, the integration was applied to an existing SSH activity detection use case.

The original Project 2 brute-force correlation rule was rule **100002**, which was designed to identify five or more failed login attempts from the same source IP within a five-minute period. Testing with more than 15 real failed login attempts and additional replayed events did not reliably trigger the rule. Rather than treating the rule as successfully validated, an existing SSH activity rule that reliably generated alerts was selected for the enrichment demonstration.

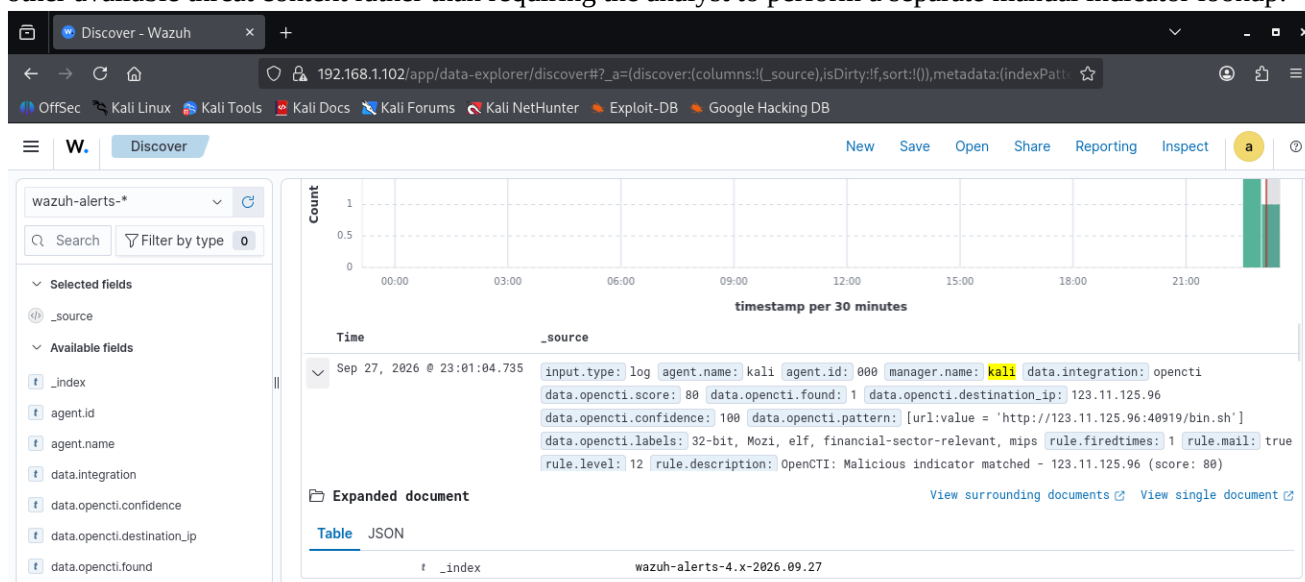
The selected rule was **100303**, which detected SSH remote-service activity and mapped the activity to MITRE ATT&CK technique **T1021.004**.

The `custom-opencti.py` integration was extended with an SSH-specific processing branch. When an alert belonging to the SSH group was received, the script extracted the source IP address from the alert's `srcip` field, with a fallback regular-expression method against the raw event when required.

The extracted source IP was then queried against OpenCTI using the same enrichment process established for network-connection events.



Following enrichment, a known malicious indicator could cause the alert to be escalated to the OpenCTI malicious-indicator rule. The resulting alert included the threat score, confidence value, associated labels, and other available threat context rather than requiring the analyst to perform a separate manual indicator lookup.

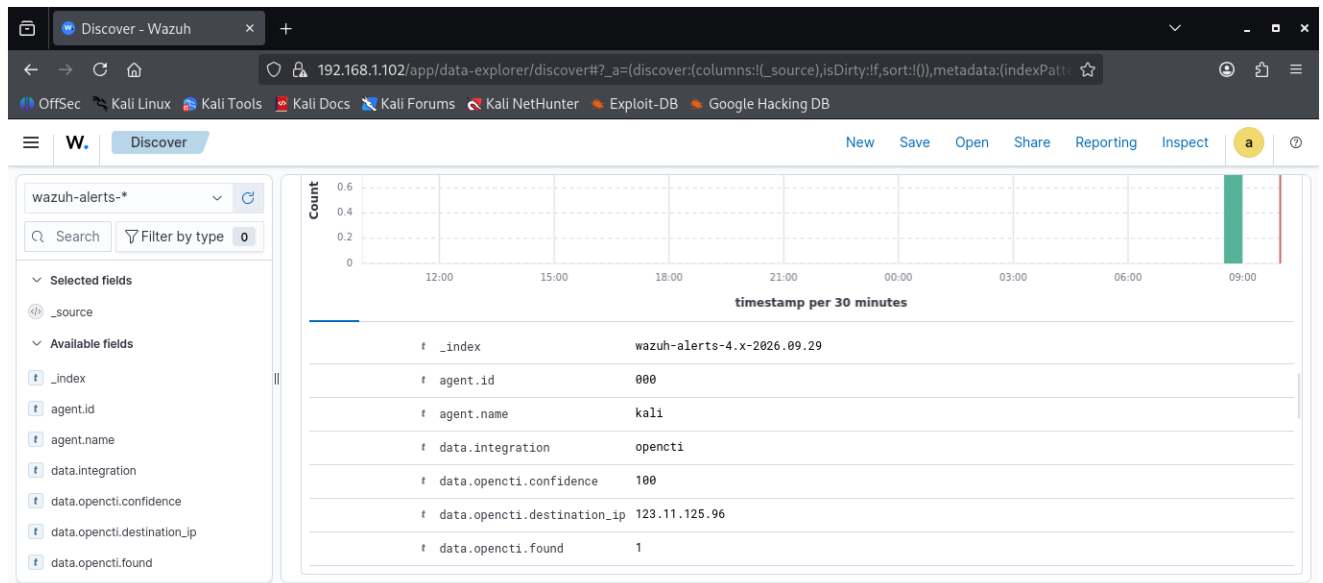


End-to-End Testing and Validation

The completed enrichment pipeline was validated using a live outbound connection from the Kali host to the known malicious URLhaus indicator **123.11.125.96** on port **40919**. The connection was made to the `/bin.sh` resource, and the resulting network activity generated the Wazuh event required to trigger the enrichment workflow.

The integration extracted the destination IP, queried OpenCTI, identified the indicator as malicious, and returned the associated threat-intelligence information to Wazuh.

The resulting alert was generated under rule **100152** at **level 12**. The enrichment contained an OpenCTI score of **80**, confidence of **100**, and labels including **Mozi**, **ELF**, **MIPS**, and **financial-sector-relevant**. The alert also contained MITRE ATT&CK information and was confirmed in the Wazuh Security Events view.



The SSH enrichment branch was subsequently validated using an existing rule-100303 alert and the known malicious indicator IP. The test confirmed that the SSH branch produced the same OpenCTI enrichment behavior, demonstrating that the pipeline could apply threat-intelligence context to more than one type of Wazuh alert.

Methodology Summary

The completed methodology established a full intelligence-driven detection workflow. External intelligence was first ingested into OpenCTI, where it was stored alongside MITRE ATT&CK information. Wazuh then generated security events from monitored activity, while the custom integration automatically extracted relevant indicators and queried OpenCTI through GraphQL.

When a known malicious indicator was identified, the returned threat context was passed back into Wazuh and converted into a dedicated high-severity alert. The completed pipeline was subsequently applied to an existing SSH detection use case, demonstrating how threat intelligence could be incorporated directly into the SOC's alerting workflow rather than being treated as a separate manual research activity.

Findings

OpenCTI Successfully Centralized Threat Intelligence

The implementation demonstrated that OpenCTI could successfully serve as a centralized threat-intelligence layer within the existing SOC environment. Intelligence from URLhaus and AlienVault OTX was ingested into the platform and made available for correlation with indicators, threat information, and MITRE ATT&CK context.

The MITRE ATT&CK integration further expanded the intelligence available within OpenCTI, providing structured relationships between threat intelligence and adversary techniques. This allowed the project to move beyond storing isolated indicators and instead provide contextual information that could support investigation and analysis.

The top screenshot displays the 'Indicators | Observations' view in the OpenCTI interface. The URL bar shows the path: `http://localhost:8080/dashboard/observations/indicators/4f331c06-c2cc-4af9-bfaa-53d4794d09db/overview`. The interface includes a search bar, a sidebar with navigation icons, and a main content area. The indicator pattern is `[url:value = 'http://210.208.110.130:51661/i']`. The valid from date is September 23, 2026 at 11:08:36 AM, and the valid until date is October 22, 2026 at 4:51:26 PM. The score is 80 / 100, and the detection status is 'No'. The author is 'Abuse.Ch', and the processing status is 'disabled'. The distribution of opinions is shown as a radar chart with values: strongly-disagree, disagree, neutral, agree, and strongly-agree. The platform creation date is September 23, 2026 at 11:22:17 AM, and the creator is 'Admin'. The standard STIX ID is `indicator--12601fb2-993d-5a5c-9e6b-...`.

The bottom screenshot displays the 'Knowledge Search | Adva' view in the OpenCTI interface. The URL bar shows the path: `http://localhost:8080/dashboard/threats/intrusion_sets/0735d024-b1ae-4e4e-8aa0-e18f84b1eb8c/knowledge`. The interface includes a search bar, a sidebar with navigation icons, and a main content area. The kill chain is set to 'mitre-attack'. The search results are displayed in a grid of techniques categorized by kill chain phases: reconnaissance (12 techniques), resource-development (9 techniques), initial-access (11 techniques), execution (24 techniques), persistence (25 techniques), privilege-escalation (14 techniques), and steal (33 techniques). The techniques are listed in a table with columns for technique name, count, and status. The table includes techniques such as 'Gather Victim Org Information', 'Acquire Access', 'Content Injection', 'Cloud Administration Command', 'Cloud Application Integration', 'Path Interception', 'Build Image on I', 'Query Public AI Services', 'Develop Capabilities', 'Hardware Additions', 'Component Object Model', 'Exclusive Control', and 'Debugger Evasic'.

External Intelligence Could Be Used Directly in SIEM Alerting

A key finding from the project was that threat intelligence stored in OpenCTI could be queried dynamically during Wazuh alert processing. The custom integration successfully extracted an indicator from a security event, queried OpenCTI through GraphQL, and returned the matching intelligence to Wazuh.

This demonstrated that OpenCTI did not have to function as a separate intelligence repository requiring manual analyst lookups. Its data could be incorporated directly into the existing SIEM alert workflow.

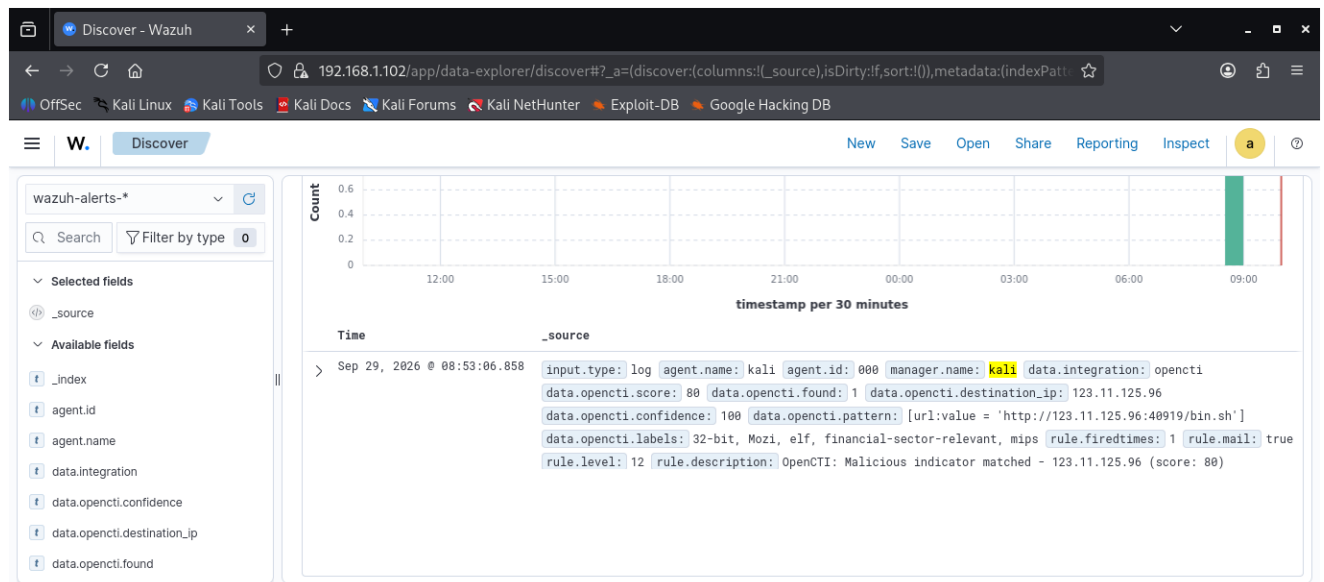
The successful test using the URLhaus indicator **123.11.125.96** confirmed this behavior. Wazuh generated an enriched level-12 alert containing the indicator's OpenCTI score, confidence value, and associated labels.

Enrichment Added Context to Otherwise Generic Network Activity

Before enrichment, a network connection primarily provided technical information about the connection itself, such as the relevant IP address and network activity. After OpenCTI enrichment, the same event could contain additional intelligence about the indicator.

The validated indicator returned a score of **80** and confidence of **100**, together with labels including **Mozi**, **ELF**, **MIPS**, and **financial-sector-relevant**. This provided the analyst with additional context at the point of alert review rather than requiring a separate investigation of the IP address.

The finding demonstrates the practical difference between detecting an event and detecting an event with associated threat intelligence.



The Enrichment Pipeline Worked With Different Wazuh Alert Sources

The implementation also demonstrated that the enrichment mechanism was not restricted to a single event source. The pipeline initially used the Sysmon network-event structure but was subsequently extended to process Linux auditd network events.

A separate SSH processing branch was also added to `custom-opencti.py`. This allowed an existing SSH activity alert to be passed through the same OpenCTI enrichment process.

The ability to process these different alert structures demonstrated that the integration could be adapted to different telemetry sources while maintaining the same underlying OpenCTI lookup workflow.

Threat Intelligence Improved the Context of the Existing SSH Detection

The application of OpenCTI enrichment to the existing SSH activity detection demonstrated an additional benefit of the integration. The original rule identified SSH remote-service activity and provided the source IP and other event information.

After enrichment, a matching malicious source could additionally provide OpenCTI intelligence such as confidence, score, threat labels, and associated ATT&CK context. This reduced the amount of separate manual threat-intelligence lookup required during initial alert triage.

The underlying SSH detection remained the same; the principal change was the addition of threat-intelligence context to the alert.

OpenCTI Revealed Intelligence Not Identified During Manual Research

The end-to-end URLhaus test produced an additional finding through OpenCTI enrichment. The returned indicator information contained **Mozi-related labels**, identifying the indicator with the Mozi malware family and characteristics associated with ELF/MIPS-based systems.

This demonstrated the value of using a structured threat-intelligence platform rather than relying exclusively on manually researched threat actors and techniques. The enrichment process was able to return contextual information associated with the specific indicator at the time it was detected.

The Project Demonstrated a Feed-to-Detection Workflow

The completed implementation established a complete operational chain:

External Threat Intelligence → OpenCTI → Indicator Matching → Wazuh Detection → CTI Enrichment → Analyst Alert

The project therefore demonstrated more than the standalone deployment of OpenCTI or the ingestion of external feeds. The intelligence was connected to an operational detection workflow and used to change the information available to the analyst during alert investigation.

The Project Demonstrated the Practical Value of Structured CTI

The overall finding from the implementation was that structured threat intelligence can add meaningful context to SIEM detections when it is integrated directly into the alert-processing workflow.

The project showed that an indicator can move through the complete process from external intelligence ingestion, through OpenCTI storage and correlation, to automated matching against Wazuh telemetry and presentation as an enriched SOC alert. This provided a practical demonstration of how threat intelligence can support detection and triage within a SOC environment.

Gaps / Challenges

The implementation was completed successfully, but several technical challenges affected the deployment and testing process. Most of these challenges were related to resource constraints, differences between Windows and Linux telemetry, Wazuh integration behavior, and the additional configuration required to convert threatintelligence matches into usable security alerts.

Limited System Resources

The primary limitation was the available memory on the Kali virtual machine. The environment allocated approximately 9 GB of RAM to Kali while simultaneously supporting Wazuh and the OpenCTI Docker stack. OpenCTI itself required several resource-intensive services, particularly Elasticsearch, which was configured with a default 2 GB heap.

Running Wazuh, OpenCTI, pfSense, and the Windows 11 virtual machine concurrently resulted in significant memory pressure. At several points, available memory dropped below 500 MB while swap usage approached the available 2.9 GB. This caused the OpenCTI Elasticsearch container to become unhealthy during startup and prevented the complete stack from operating reliably alongside the other lab components.

The limitation was addressed operationally rather than by rebuilding the laboratory. OpenCTI and Wazuh were run separately when necessary, and services were started and stopped according to the testing requirement. This allowed the project to proceed without requiring an additional virtual machine.

Differences Between Sysmon and Linux Audit Telemetry

The initial enrichment design was based on Windows Sysmon network connection events, where the destination IP is available through structured event fields. However, the available system resources made it impractical to operate the Windows VM, Wazuh, OpenCTI, and the other infrastructure components simultaneously.

Testing was therefore moved to the Kali endpoint using Linux auditd. This introduced a difference in the structure of the telemetry available to the custom integration. Unlike Sysmon, auditd did not provide the required network information in the same structured JSON field. The relevant connection information was instead contained within the raw event message.

The integration therefore required separate extraction logic: the Sysmon branch used the structured destination IP field, while the auditd branch used regular-expression matching against the raw event content to extract the destination IP and port.

Auditd Monitoring Rules Were Not Persistent

The auditd network connection rule was initially added dynamically using `auditctl`. Such rules are maintained by the running audit subsystem and are not automatically persistent across system restarts.

As a result, an apparently non-functional test could be caused by the monitoring rule no longer being loaded rather than by a problem with the Wazuh integration or detection logic. This required verification that the `network_connect` audit rule was active before testing.

A persistent configuration using the audit rules directory was identified as a future improvement, but the project proceeded with the dynamically applied rule during implementation and validation.

Wazuh Rule Matching and Alert Generation

Another challenge was ensuring that auditd events were not only decoded by Wazuh but also generated alerts that could trigger the integration. Wazuh successfully decoded the auditd events, but the initial custom rule did not produce the expected alert because the built-in auditd rule hierarchy already handled the events at level 0.

The custom rule was subsequently positioned as a child of the existing auditd rule and restricted to the `network_connect` audit key. This produced a level-5 alert that could then be passed to the custom OpenCTI integration.

Custom Integration Registration

The custom OpenCTI integration initially failed to load because Wazuh did not recognize `opencti` as a supported integration name. The Wazuh integration service uses predefined integration names unless custom integrations are registered using the appropriate naming convention.

The integration was therefore renamed to `custom-opencti`, and the associated configuration and script references were updated. This allowed Wazuh to invoke the custom Python integration normally.

This issue demonstrated that successful script development alone was not sufficient; the integration also had to conform to Wazuh's integration service requirements.

Custom Integration Registration

The custom OpenCTI integration initially failed to load because Wazuh did not recognize `opencti` as a supported integration name. The Wazuh integration service uses predefined integration names unless custom integrations are registered using the appropriate naming convention.

The integration was therefore renamed to `custom-opencti`, and the associated configuration and script references were updated. This allowed Wazuh to invoke the custom Python integration normally.

This issue demonstrated that successful script development alone was not sufficient; the integration also had to conform to Wazuh's integration service requirements.

OpenCTI Enrichment Did Not Automatically Create an Alert

A significant implementation issue occurred after the integration successfully queried OpenCTI and found a malicious indicator. The script reported that the match had been successfully sent through the Wazuh Unix socket, but no corresponding security alert appeared in the dashboard.

The issue was that sending an enrichment message through the Wazuh socket did not, by itself, create the required alert. A dedicated local Wazuh rule was required to decode the JSON message, identify the OpenCTI integration field, and assign an appropriate alert level.

Rules 100150–100152 were subsequently implemented for the OpenCTI integration. The malicious-indicator rule generated the level-12 alert used during final validation.

Existing SSH Brute-Force Rule Did Not Fire Reliably

The existing SSH brute-force rule, rule 100002, was intended to detect repeated failed authentication attempts from the same source IP within a defined frequency and timeframe. During testing, the rule did not reliably generate an alert despite more than 15 failed login attempts and additional replayed events through `wazuhlogtest`.

Rather than modifying the rule without sufficient evidence about the underlying correlation behavior, the project retained the existing rule as a documented limitation and selected the reliably firing SSH remote-service rule 100303 for the applied detection use case.

The OpenCTI enrichment was then extended to the SSH alert path, allowing the source IP associated with rule 100303 to be checked against OpenCTI intelligence.

Third Threat-Intelligence Feed Not Added

ThreatFox was considered as an additional intelligence source during feed selection. However, URLhaus and AlienVault OTX already satisfied the external-feed requirement, while the available system resources were already being heavily consumed by OpenCTI and its supporting services.

Adding another feed would therefore have increased resource and ingestion demands without being necessary for the project's core objectives. ThreatFox was consequently not included in the final deployment.

This represents a scope limitation rather than a failure of the implementation. The final environment still provided multiple external intelligence sources together with MITRE ATT&CK data.

Operational Dependency on Service Sequencing

Because of the memory limitations, the final laboratory environment depended on careful sequencing of Wazuh and OpenCTI services. In normal operation, both platforms would ideally remain available simultaneously so that enrichment could occur continuously.

Within the available hardware, however, running both environments together was not consistently reliable. The project therefore used controlled start/stop procedures when switching between OpenCTI activities and Wazuh testing.

A related Wazuh manager startup timeout was also encountered and addressed by increasing the `systemd TimeoutStartSec` value to 300 seconds. This prevented the manager from being terminated during longer startup operations..

Overall Impact of the Challenges

The challenges did not prevent completion of the core project objectives. Instead, they required adaptations to the original implementation approach, particularly the switch from Windows Sysmon-based testing to Linux auditd, controlled operation of resource-intensive services, and additional Wazuh rules for processing enrichment results.

The final implementation successfully demonstrated the complete workflow from external threat intelligence ingestion through OpenCTI, indicator matching through the custom integration, and presentation of enriched intelligence within Wazuh. The limitations primarily affected operational efficiency, scalability, and the breadth of the laboratory environment rather than the core functionality demonstrated by the project.

6. Recommendations / Next Steps

The completed implementation demonstrated that OpenCTI can provide actionable threat-intelligence context to Wazuh alerts. The following recommendations focus on improving the reliability, scalability, persistence, and operational usefulness of the environment.

Increase Available System Resources

The most significant improvement would be to provide additional memory for the OpenCTI and Wazuh environments. OpenCTI's Elasticsearch component was the primary resource consumer, and the available 9 GB of RAM required Wazuh and OpenCTI to be operated separately during several stages of testing.

Increasing the available memory, or moving OpenCTI to a dedicated virtual machine or host, would allow the SIEM and threat-intelligence platform to operate concurrently. This would make the enrichment pipeline more representative of a continuously operating SOC environment.

Reduce Elasticsearch Resource Consumption Where Appropriate

Where additional hardware is not immediately available, OpenCTI's Elasticsearch resource allocation could be reviewed and reduced to a level appropriate for the laboratory workload. This should be done carefully because excessive reduction could negatively affect OpenCTI stability and performance.

Resource monitoring should be performed after any adjustment to ensure that OpenCTI, Wazuh, and their supporting services remain stable under normal laboratory activity.

Make Auditd Network Monitoring Persistent

The current auditd `network_connect` rule was applied dynamically during testing. A future implementation should place the required rule in the persistent audit rules configuration so that it is automatically restored after a system restart.

This would eliminate the need to manually reapply the rule before testing and would make the Linux endpoint more suitable for continuous monitoring.

Restore Continuous Windows Sysmon Testing

The original enrichment design included Windows Sysmon network events, but resource constraints required most validation to be performed through Linux auditd. With additional system resources, the Windows endpoint should be brought back into the normal testing workflow.

This would allow the Sysmon branch of `custom-opencti.py` to be tested against live Windows telemetry and would provide a more complete demonstration of cross-platform threat-intelligence enrichment.

Resolve the SSH Brute-Force Correlation Rule

The existing SSH brute-force rule, 100002, should be investigated further to determine why the expected frequency/timeframe correlation was not consistently producing an alert.

Future testing should focus on Wazuh's analysisd correlation state, event grouping, rule dependencies, and the exact conditions required for frequency-based rules to trigger. Once the underlying issue is understood, the rule can be corrected and independently validated rather than bypassed.

Expand Threat-Intelligence Sources

ThreatFox was considered but excluded because of the resource limitations and because URLhaus and AlienVault OTX already provided sufficient external intelligence for the project.

Once additional resources are available, ThreatFox or another relevant intelligence source could be added. Additional feeds should be selected based on the type and quality of intelligence they provide rather than simply increasing the number of feeds.

The expanded environment could then compare indicators from different sources and provide analysts with broader coverage and additional confidence context.

Improve Persistence and Operational Reliability

The current environment required controlled service sequencing because of memory constraints. A more mature implementation should allow Wazuh and OpenCTI to remain operational simultaneously, with the enrichment integration available continuously.

This would move the laboratory from a demonstration-oriented setup toward a more realistic SOC architecture in which a Wazuh alert can be enriched immediately whenever a matching indicator is observed.

Expand Enrichment Beyond IP Addresses

The current custom integration primarily uses IP addresses as the matching field. Future development could extend the enrichment logic to additional observable types, such as domains, URLs, file hashes, and other supported indicators.

This would increase the number of Wazuh detections that could benefit from OpenCTI context and make the integration more useful across different categories of security events.

Expand the Context Returned to Analysts

The current enrichment already provides useful information including confidence, score, labels, and MITRE ATT&CK context. Future iterations could return additional structured information such as the associated threat actor, campaign, malware family, source feed, indicator type, and relevant relationships.

Presenting this information directly in the Wazuh alert would reduce the amount of manual investigation required after an indicator match.

Establish a More Formal Indicator Validation Process

Future testing should use a consistent validation process for each intelligence source and observable type. This could include recording the indicator source, ingestion time, confidence, expected OpenCTI relationship, Wazuh event that should trigger enrichment, and the resulting alert.

Such a process would make future feed changes and integration updates easier to validate and troubleshoot.

Maintain Separation Between Threat Intelligence Research and Technical Validation

The case study demonstrated how threat intelligence can be applied to understand the threat landscape surrounding a specific organization and sector. Future iterations should continue to distinguish this type of intelligence research from the technical validation of the Wazuh–OpenCTI integration.

Maintaining this separation ensures that documented threat actors, historical incidents, and sector relationships are clearly presented as research findings, while live indicator matching and alert enrichment remain part of the technical implementation.

Long-Term Development

The completed project provides a foundation for further development of an intelligence-driven SOC workflow. A future version could incorporate persistent multi-source feeds, broader observable matching, continuous Wazuh/OpenCTI availability, improved correlation rules, and richer automated enrichment.

The long-term objective would be to reduce the amount of manual context gathering required during alert triage while maintaining clear visibility into the intelligence source and evidence supporting each enrichment result.

Conclusion

This project successfully demonstrated the integration of OpenCTI with an existing Wazuh-based Security Operations Center laboratory to introduce threat-intelligence context into security monitoring and alert investigation. The implementation extended the existing SOC environment beyond conventional event detection by allowing observed network activity to be compared against externally sourced threat intelligence.

OpenCTI was deployed within the existing laboratory environment and connected to URLhaus and AlienVault OTX, while MITRE ATT&CK provided additional threat actor, malware, and technique context. The resulting intelligence was used to establish a centralized source of threat information that could be queried by the Wazuh environment.

A custom Python integration, `custom-opencti.py`, was developed to connect Wazuh alerts with OpenCTI through its GraphQL interface. The integration extracted relevant IP information from monitored events, queried OpenCTI for matching indicators, and returned the available intelligence to Wazuh. Additional Wazuh rules were implemented to convert the returned enrichment data into analyst-visible security alerts.

The final end-to-end validation demonstrated that a known malicious URLhaus indicator could be detected and enriched automatically. The resulting Wazuh alert included a high-severity OpenCTI match with a score of 80, confidence of 100, and associated labels including Mozi and financial-sector relevance. This demonstrated that external threat intelligence could be incorporated directly into the alert-triage workflow rather than requiring analysts to perform a separate manual lookup.

The project also applied the enrichment capability to an existing SSH detection use case. Although the original SSH brute-force correlation rule did not reliably trigger during testing, the reliably firing SSH remote-service activity rule was used to validate the enrichment process. This demonstrated how existing SIEM detections can be enhanced with threat-intelligence context without replacing their underlying detection logic.

Several implementation challenges were encountered, particularly limited system resources, differences between Sysmon and auditd telemetry, Wazuh integration requirements, alert-rule dependencies, and the need

for additional debugging visibility. These challenges required changes to the implementation approach but did not prevent successful validation of the core workflow.

Overall, the project established a functional threat-intelligence enrichment pipeline from external intelligence sources through OpenCTI and into Wazuh. It demonstrated how structured cyber threat intelligence can provide additional context during alert triage, support more informed investigation, and connect individual security events with broader threat intelligence. The resulting environment provides a practical foundation for further development, including continuous Wazuh–OpenCTI operation, persistent endpoint monitoring, additional intelligence feeds, broader indicator matching, and richer automated enrichment.

8. Appendix

This appendix contains the principal configuration files, scripts, detection rules, and validation commands used during the implementation of the Wazuh–OpenCTI threat-intelligence enrichment pipeline. Only the final working configurations and key validation evidence are included.

Appendix A — Wazuh–OpenCTI Integration

A.1 Wazuh Integration Wrapper

The following wrapper script was used to register and execute the custom OpenCTI integration through the Wazuh integration framework.

```
#!/bin/sh
WPYTHON_BIN="framework/python/bin/python3"
SCRIPT_PATH_NAME="$0"
DIR_NAME="$(cd $(dirname ${SCRIPT_PATH_NAME}); pwd -P) "
SCRIPT_NAME="$(basename ${SCRIPT_PATH_NAME}) "

case ${DIR_NAME} in
    */active-response/bin | */wodles*)
        if [ -z "${WAZUH_PATH}" ]; then
            WAZUH_PATH="$(cd ${DIR_NAME}/../..; pwd) "
        fi
        PYTHON_SCRIPT="${DIR_NAME}/${SCRIPT_NAME}.py"
        ;;
    */bin)
        if [ -z "${WAZUH_PATH}" ]; then
            WAZUH_PATH="$(cd ${DIR_NAME}/..; pwd) "
        fi
        PYTHON_SCRIPT="${WAZUH_PATH}/framework/scripts/${echo ${SCRIPT_NAME} | sed
's/\-/_/g').py"
        ;;
esac
```

```

*/integrations)
    if [ -z "${WAZUH_PATH}" ]; then
        WAZUH_PATH="$(cd ${DIR_NAME}/..; pwd)"
fi
    PYTHON_SCRIPT="${DIR_NAME}/${SCRIPT_NAME}.py"
;; esac

${WAZUH_PATH}/${WPYTHON_BIN} ${PYTHON_SCRIPT} "$@"

```

A.2 Custom OpenCTI Enrichment Script

The following is the final working version of `custom-opencti.py`. It handles IP extraction from Sysmon, auditd, and SSH events, queries OpenCTI through GraphQL, and returns matching threatintelligence information to Wazuh.

```

#!/var/ossec/framework/python/bin/python3
import json import os import re import
sys import ipaddress
from socket import AF_UNIX, SOCK_DGRAM, socket
try:
    import requests except Exception:    print("No module 'requests'
found. Install: pip install requests")    sys.exit(1)

OPENCTI_URL = "http://localhost:8080/graphql"
pwd =
os.path.dirname(os.path.dirname(os.path.realpath(__file__)))
LOG_FILE = f'{pwd}/logs/integrations.log'
SOCKET_ADDR = f'{pwd}/queue/sockets/queue'

debug_enabled = False

def debug(msg):    if debug_enabled:
with open(LOG_FILE, 'a') as f:
    f.write(str(msg) + '\n')

def
main(args):
    global debug_enabled
    if len(args) >= 4:
        debug_enabled = len(args) > 4 and args[4] == 'debug'
    else:
        sys.exit(2)

    alert_file_location = args[1]
    api_key = args[2]
    json_alert = get_json_alert(alert_file_location)
    debug(f"# Processing alert: {json_alert.get('id')}")
    msg = process_alert(json_alert,
api_key)    if msg:
        send_msg(msg, json_alert.get('agent'))
    def get_json_alert(file_location):
try:        with open(file_location)
as f:

```

```

        return json.load(f)
except Exception as e:
    debug(f"# Error reading alert file: {e}")
sys.exit(6)
def process_alert(alert,
api_key):
    dest_ip = None
    groups = alert.get('rule', {}).get('groups', [])
    if 'sysmon_event3' in
groups:
        win = alert.get('data', {}).get('win', {}).get('eventdata', {})
dest_ip = win.get('destinationIp')
is_ipv6 =
win.get('destinationIsIpv6', 'false')
if is_ipv6 == 'true':
dest_ip = None
    elif alert.get('decoder', {}).get('name') == 'auditd' and \
alert.get('data', {}).get('audit', {}).get('key') ==
'network_connect':
        full_log = alert.get('full_log', '')
        match = re.search(r'saddr_fam=inet laddr=([\d.]+) lport=(\d+)', full_log)
if match:
        dest_ip = match.group(1)
    elif 'sshd' in
groups:
        dest_ip = alert.get('data', {}).get('srcip')
if not dest_ip:
        full_log = alert.get('full_log', '')
        match = re.search(r'\b(\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3})\b',
full_log)
        if match:
            dest_ip = match.group(1)
        if not dest_ip:
            debug("# No usable
destination IP found")
            return None
        try:
            if not
ipaddress.ip_address(dest_ip).is_global:
                debug(f"# {dest_ip} is not a public IP, skipping")
return None
        except ValueError:
            return None
        debug(f"# Querying OpenCTI for IP:
{dest_ip}")
        result = query_opencti(dest_ip,
api_key)

    alert_output = {
        'integration': 'opencti',
        'opencti': {
            'found': 0,
            'destination_ip': dest_ip
        }
    }
    if
result:
        alert_output['opencti'].update({
            'found': 1,
            'score': result.get('x_opencti_score'),
            'confidence': result.get('confidence'),
            'pattern': result.get('pattern'),
            'labels': result.get('objectLabel', []),
        })

```

```

        debug(f"# MATCH FOUND, sending: {alert_output}")
return alert_output

        debug(f"# No match found in OpenCTI for {dest_ip}")
return None

        def query_opencti(ip,
api_key):    query = """
query($search: String) {
    indicators(search: $search, first: 5) {
edges {
            node {
                pattern
confidence
                x_opencti_score
objectLabel { value }
            }
        }
    }
}
"""

        headers = {
            "Authorization": f"Bearer {api_key}",
            "Content-Type": "application/json"
        }
try:
        resp = requests.post(
            OPENCTI_URL,
            json={"query": query, "variables": {"search": ip}},
headers=headers,
            timeout=10
        )
        if
resp.status_code != 200:
            debug(f"# OpenCTI API error: {resp.status_code}")
return None

        data = resp.json()
        edges = data.get('data', {}).get('indicators', {}).get('edges', [])
        for edge in
edges:
            node = edge['node']
            if ip
in (node.get('pattern') or ''):
                node['objectLabel'] = [
                    l['value'] for l in node.get('objectLabel', [])
                ]
return node

        return None
    except Exception as
e:
        debug(f"# Exception querying OpenCTI: {e}")
return None

        def send_msg(msg,
agent=None):
            debug(f"# Raw string being sent: {msg}")
            if not agent or agent.get('id') ==
'000':

```

```

        string = '1:opencti:{0}'.format(json.dumps(msg))
else:
    location = '[{0}] ({1}) {2}'.format(
agent['id'],
        agent['name'],
agent.get('ip', 'any')
    )
    location = location.replace('|', '||').replace(':', '|:')
string = '1:{0}->opencti:{1}'.format(
        location,
json.dumps(msg)
    )
try:
    sock = socket(AF_UNIX, SOCK_DGRAM)
sock.connect(SOCKET_ADDR)
    sock.send(string.encode())
sock.close()
    debug("# Socket send completed successfully")
except
FileNotFoundError:
    debug(f"# Error: Unable to open socket at {SOCKET_ADDR}")
sys.exit(5)
if __name__ ==
'__main__':
    main(sys.argv)

```

A.3 Wazuh Integration Configuration

The following configuration registers the custom integration with Wazuh and specifies the alert groups that can trigger it.

```

<integration>
  <name>custom-opencti</name>
  <api_key>YOUR_OPENCTI_TOKEN</api_key>
  <group>sysmon_event3,network_connect,ssh_brute_force,sshd</group>
  <alert_format>json</alert_format>
</integration>

```

Appendix B — Auditd Monitoring Configuration

B.1 Wazuh Audit Log Collection

The following `localfile` configuration enables Wazuh to monitor the Linux audit log.

```

<localfile>
  <log_format>audit</log_format>
  <location>/var/log/audit/audit.log</location>
</localfile>

```

B.2 Auditd Network Connection Rule

The following audit rule was used to capture outbound `connect()` system calls and associate them with the `network_connect` key. `sudo auditctl -a exit,always -F arch=b64 -S connect -k network_connect`

Note: This rule was applied dynamically during the project and was not persistent across system reboots.

B.3 Wazuh Rule for Auditd Network Connections

The following custom rule converted the relevant auditd events into Wazuh alerts that could trigger the OpenCTI integration.

```
<group name="local,">
  <rule id="100100" level="5">
    <if_sid>80700</if_sid>
    <field name="audit.key">network_connect</field>
    <description>Outbound network connection detected (auditd)</description>
    <group>network_connect,</group>
  </rule>
</group>
```

Appendix C — OpenCTI Enrichment Detection Rules

The following rules process the message returned by the custom OpenCTI integration and generate the corresponding Wazuh alerts.

```
<group name="opencti,">
  <rule id="100150" level="0">
    <decoded_as>json</decoded_as>
    <field name="integration">opencti</field>
    <description>OpenCTI integration messages.</description>
    <options>no_full_log</options>
  </rule>

  <rule id="100151" level="3">
    <if_sid>100150</if_sid>
    <field name="opencti.found">0</field>
    <description>OpenCTI: No match found for
$(opencti.destination_ip)</description>
    <options>no_full_log</options>
  </rule>

  <rule id="100152" level="12">
    <if_sid>100150</if_sid>
    <field name="opencti.found">1</field>
```

```
        <description>OpenCTI: Malicious indicator matched -  
$(opencti.destination_ip) (score: $(opencti.score))</description>  
        <mitre>  
            <id>T1071</id>  
        </mitre>  
        <options>no_full_log</options>  
    </rule>  
</group>
```

Appendix D — End-to-End Validation

The final integration was validated using a known malicious URLhaus indicator. The connection generated network telemetry, triggered the Wazuh detection and OpenCTI enrichment workflow, and was then checked for the resulting level-12 alert.

```
curl -s -m 5 http://123.11.125.96:40919/bin.sh -o /dev/null -w "HTTP code:  
${http_code}\n"  
    sleep  
5
```

```
sudo grep -a '"id":"100152"' /var/ossec/logs/alerts/alerts.json | tail -1
```

The resulting alert confirmed the OpenCTI match and included the associated indicator score and threat-intelligence context.

Appendix E — Architectural Diagram

